RV UNIVERSITY
School of Computer Science and Engineering
Bengaluru – 560059



Essentials of Internet of Things
COURSE CODE: CS3182

IV Semester
Laboratory Record

Name	**Esheshwai Kumari**
USN	**1RUA24BCA0025**
Academic Year	**2026 - 2027**

RV UNIVERSITY

School of Computer Science and Engineering

Bengaluru – 560 059



LABORATORY CERTIFICATE

This is to certify that **Mr./Ms.** Esheshwari Kumari
has satisfactorily completed the course of activities in **Essentials of Internet of Things** prescribed by the **School of Computer Science and Engineering** during the year **2025-26.**

Name of the Candidate: Esheshwari Kumari

USN: 1RUA24BCA0025 **Semester: 04**

Marks	
Maximum	**Obtained**
10	

Signature of Faculty in-charge **Program Director**

Date: 20 AUGUST 2026

Vision and Mission of the School of Computer Science and Engineering

Vision

To be a pioneering school of Computer Science and Engineering committed to fostering liberal education and empowering the next generation of technologists to make a positive global socio-economic impact.

Mission

- To be a pioneer in computer science education and benchmark ourselves with the world's top computer science and engineering institutions.
- To provide state-of-the-art facilities that enable exemplary pedagogy, advanced research, innovation and entrepreneurship in emerging technologies of computer science.
- To promote a culture of cooperation and inclusiveness among students and faculty from diverse communities enabling them to take part in interdisciplinary and multidisciplinary research, contributing to institution-building.
- To foster excellence through national and international academic, industry collaborations, bringing in diverse perspectives to drive innovation.
- To nurture a talented pool of ethical, self-driven and empathetic problem solvers to achieve sustainable development goals.

INDEX

Sl. No	Program Name.	Date	Page No.
1	Lab Activity-1A(Controlling Actuators through Serial Monitor)	03/08/26	05 - 06
2	Lab Activity-1B(Creating Different LED Patterns Using Push Buttons)	10/08/26	07 - 08
3	Lab activity-1C(Controlling Servo Motor Using Joystick)	17/08/26	09 - 11
4	Lab Activity-2(ESP32-Based Ultrasonic Distance Warning System)	24/08/26	12 - 15
5	Lab Activity-3a(DHT22-Based Temperature and Humidity Monitor)	31/08/26	16 - 18
6	Lab Activity-3b(IR-Based Object Distance Monitoring System)	31/08/26	19 - 20
7	Lab Activity-4(ESP32 LDR Light Sensor Alert System)	07/09/26	21-22
8	Lab Activity-5(Pico Relay Dual-Load Switch: Motor & Buzzer Controller)	07/09/26	23-24
9	Lab Activity-6a(Advanced burglar alarm security system)	05/10/26	25-28
10	Lab Activity-6b(Automated LED light control)	13/10/26	29-30
11	Lab Activity-7(Upload humidity & temperature data to Thing Speak)	02/11/26	31-32
11	Lab Activity-8(Controlling LEDs, relay & buzzer using Blynk app)	11/09/26	33
12	Lab Activity-9(HTTP Based: Hosting a basic server from the ESP32)	11/09/26	34-37
13	Lab Activity-10(MQTT Based: Controlling LEDs/Motors)	11/09/26	38-41
14	Lab Activity-11(MQTT Based: Displaying humidity and temperature)	11/09/26	42-45

Lab Experiment-1A

Controlling Actuators through Serial Monitor

Hardware Connections

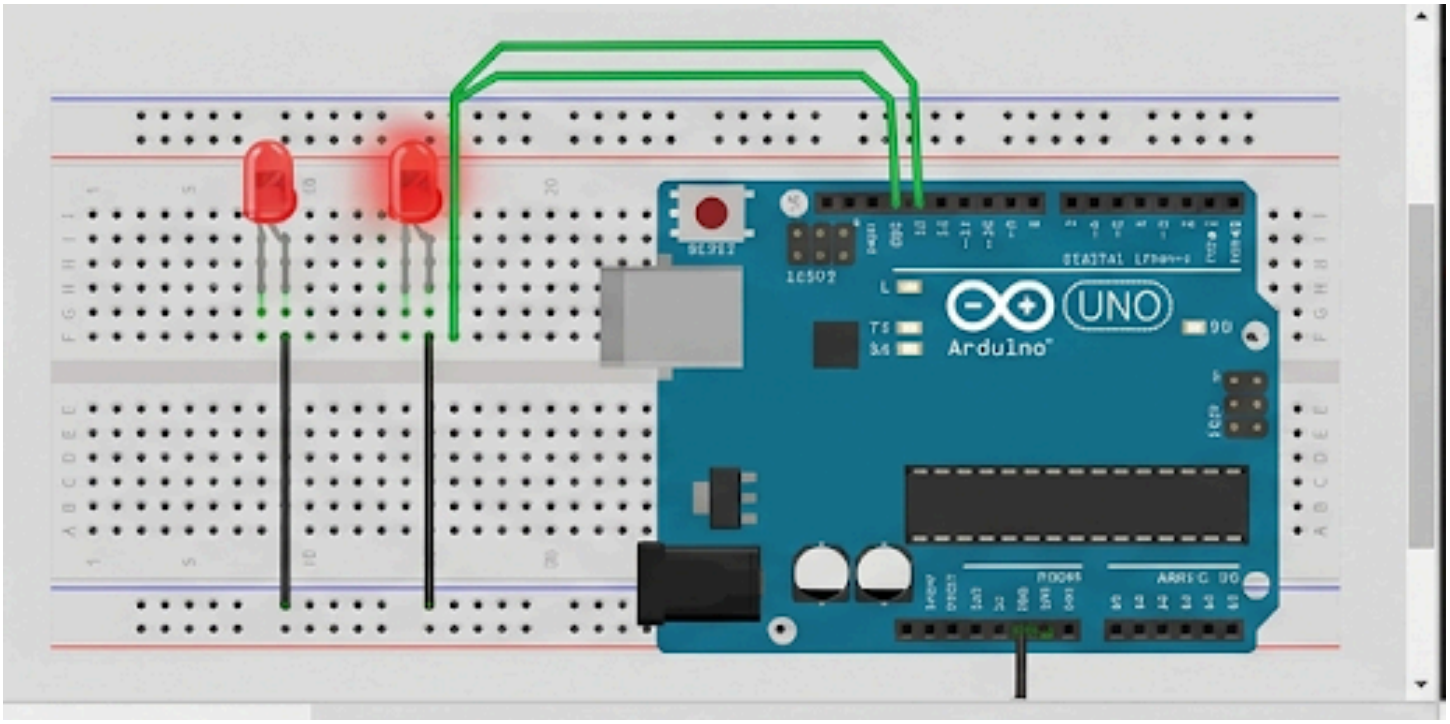
Components	**Arduino Pins**
LED Anode	D8
LED Cathode	GND
LED Anode	D9
LED Cathode	GND

Arduino Program:

```
const int led1 = 8;
const int led2 = 9;
void setup()
{
pinMode(led1, OUTPUT);
pinMode(led2, OUTPUT);
Serial.begin(9600);
Serial.println("Commands:");
Serial.println("L - LED ON");
Serial.println("l - LED OFF");
Serial.println("B - LED ON");
Serial.println("b - LED OFF");
}
void loop()
{
if(Serial.available())
{
char data = Serial.read();
switch(data)
{
case 'L':
digitalWrite(led1,HIGH);
```

```
break;
case 'l':
digitalWrite(led1,LOW);
break;

case 'B':
digitalWrite(led2,HIGH);
break;
case 'b':
digitalWrite(led2,LOW);
break;
}
}
}
```



Lab Experiment-1B

Creating Different LED Patterns Using Push Buttons

Components

- Arduino Uno
- 4 LEDs
- 2 Push Buttons

Hardware Connections

LEDs

LED	Arduino Pin
LED1	D2
LED2	D3
LED3	D4
LED4	D5

Push Buttons

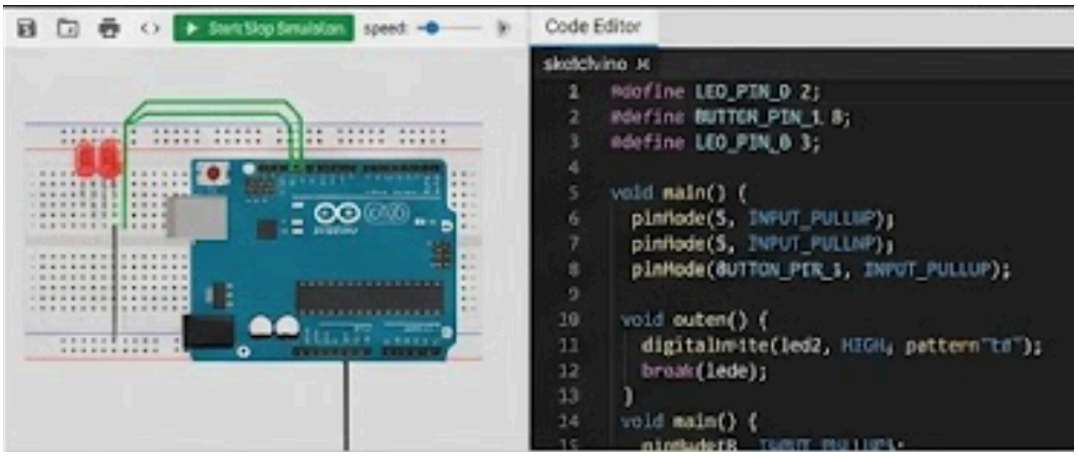
Button	Arduino Pin
Button1	D8
Button2	D9

Arduino Program

```
int leds[] = {2,3,4,5};
void setup()
{
for(int i=0;i<4;i++)

pinMode(leds[i],OUTPUT);
pinMode(8,INPUT_PULLUP);
pinMode(9,INPUT_PULLUP);
}
```

```
void loop()
{
if(digitalRead(8)==LOW)
{
// Left to Right
for(int i=0;i<4;i++)
{
digitalWrite(leds[i],HIGH);
delay(200);
digitalWrite(leds[i],LOW);
}
}
if(digitalRead(9)==LOW)
{
// Right to Left
for(int i=3;i>=0;i--)
{
digitalWrite(leds[i],HIGH);
delay(200);
digitalWrite(leds[i],LOW);
}
}
}
```



Lab Experiment-1C

Controlling Servo Motor Using Joystick

Components

- Arduino Uno
- Joystick Module
- Servo Motor ×2

Hardware Connections

Joystick Pin	Arduino Pin
VCC	5V
GND	GND
VRx	A1
VRy	A0

Servo 1

Servo Pin	Arduino pin
Signal	D8
VCC	5V
GND	GND

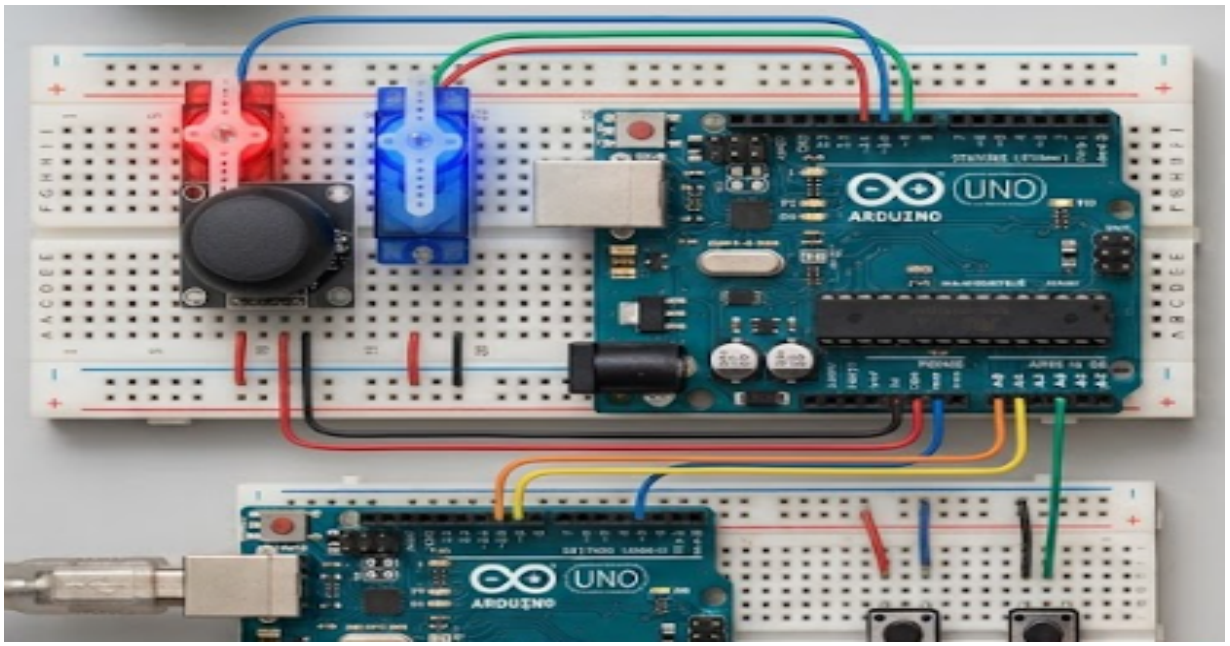
Servo 2

Servo Pin	Arduino pin
Signal	D9
VCC	5V
GND	GND

Arduino Program

```
#include <Servo.h>
Servo servo1;
Servo servo2;
const int x_key = A1;
const int y_key = A0;
int x_pos;
int y_pos;
int pos1 = 90;
int pos2 = 90;
void setup()
{
servo1.attach(8);
servo2.attach(9);
servo1.write(pos1);
servo2.write(pos2);
}
void loop()
{
x_pos = analogRead(x_key);
y_pos = analogRead(y_key);
if(x_pos<300)
pos1-=2;
else if(x_pos>700)
pos1+=2;
if(y_pos<300)
pos2-=2;
else if(y_pos>700)

pos2+=2;
pos1 = constrain(pos1,0,180);
pos2 = constrain(pos2,0,180);
servo1.write(pos1);
servo2.write(pos2);
delay(20);
}
```


UNO
ARDUINO
UNO
ARDUINO

Lab Experiment -2

ESP32-Based Ultrasonic Distance Warning System

Hardware Connections
HC-SR04 → ESP32

HC-SR04	**ESP-32**
VCG	5V
GND	GND
TRIG	GPIO5
ECHO	GPIO18

TFT ILI9341 → ESP32

TFT PIN	**ESP32 PIN**	**FUNCTION**
VCC	3.3V	Power
GND	GND	Ground
CS	GPIO 15	Chip Select
RESET	GPIO 4	Reset
DC/RS	GPIO 2	Data/Command
MOSI/SDI	GPIO 23	SPI Data
SCK/CLK	GPIO 18	SPI Clock
MISO/SDO	GPIO 19	SPI Data out
LED	3.3V	Backlight

Buzzer → ESP32

Buzzer ESP32
+ GPIO 27
− GND

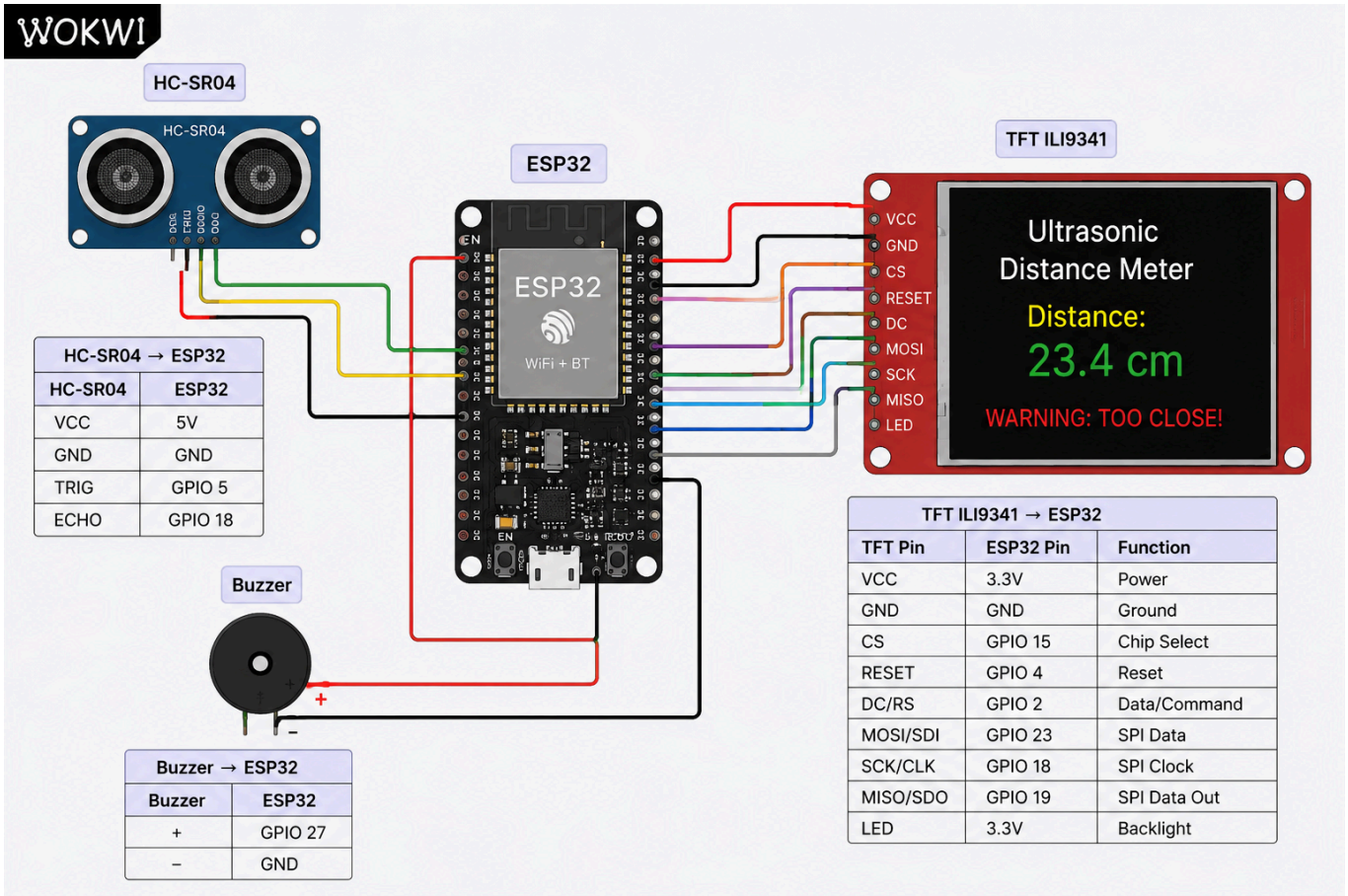
ESP32 Program

```
#include <SPI.h>
#include <Adafruit_GFX.h>
#include <Adafruit_ILI9341.h>
#define TRIG 5
#define ECHO 18
#define BUZZER 27
#define TFT_CS 15
#define TFT_DC 2

#define TFT_RST 4
Adafruit_ILI9341 tft(TFT_CS, TFT_DC, TFT_RST);
void setup() {
pinMode(TRIG, OUTPUT);
pinMode(ECHO, INPUT);
pinMode(BUZZER, OUTPUT);
tft.begin();
tft.setRotation(1);
tft.fillScreen(ILI9341_BLACK);
tft.setTextColor(ILI9341_WHITE);
tft.setTextSize(2);
tft.setCursor(30, 30);
tft.print("Ultrasonic");
tft.setCursor(30, 60);
tft.print("Distance Meter");
delay(2000);
}
void loop() {
digitalWrite(TRIG, LOW);
delayMicroseconds(2);
digitalWrite(TRIG, HIGH);
delayMicroseconds(10);
digitalWrite(TRIG, LOW);
long time = pulseIn(ECHO, HIGH);
float distance = time * 0.0343 / 2;
tft.fillRect(0, 100, 320, 100, ILI9341_BLACK);
```

```
tft.setCursor(30, 110);
tft.setTextColor(ILI9341_YELLOW);
tft.setTextSize(3);
tft.print("Distance:");
tft.setCursor(30, 155);
tft.setTextColor(ILI9341_GREEN);
tft.print(distance, 1);

tft.print(" cm");
digitalWrite(BUZZER, distance < 20);
if (distance < 20) {
tft.setCursor(30, 205);
tft.setTextColor(ILI9341_RED);
tft.setTextSize(2);
tft.print("WARNING: TOO CLOSE!");
}
delay(500);
}
```



HC-SR04 → ESP32	
HC-SR04	ESP32
VCC	5V
GND	GND
TRIG	GPIO 5
ECHO	GPIO 18

TFT ILI9341 → ESP32		
TFT Pin	ESP32 Pin	Function
VCC	3.3V	Power
GND	GND	Ground
CS	GPIO 15	Chip Select
RESET	GPIO 4	Reset
DC/RS	GPIO 2	Data/Command
MOSI/SDI	GPIO 23	SPI Data
SCK/CLK	GPIO 18	SPI Clock
MISO/SDO	GPIO 19	SPI Data Out
LED	3.3V	Backlight

Buzzer → ESP32	
Buzzer	ESP32
+	GPIO 27
−	GND

Lab Experiment - 3(a)

DHT22-Based Temperature and Humidity Monitor

Hardware Connections
DHT22 to ESP32

DHT22 Pin	ESP 32
VCC	3.3V
Data	GPIO4
GND	GND

ST7735 TFT to ESP32
For a typical SPI-based ST7735 module:

ST7735 Pin	ESP32 Pin	Function
VCC	3.3V	Power
GND	GND	Ground
SCL / SCK	GPIO 18	SPI Clock
SDA / MOSI	GPIO 23	SPI Data
CS	GPIO 5	Chip Select
DC / A0	GPIO 2	Data/Command
RST	GPIO 15	Reset
LED / BL	3.3V	Backlight

Note: ST7735 modules can have different pin names and configurations. The above connection is for a common SPI ST7735 module.

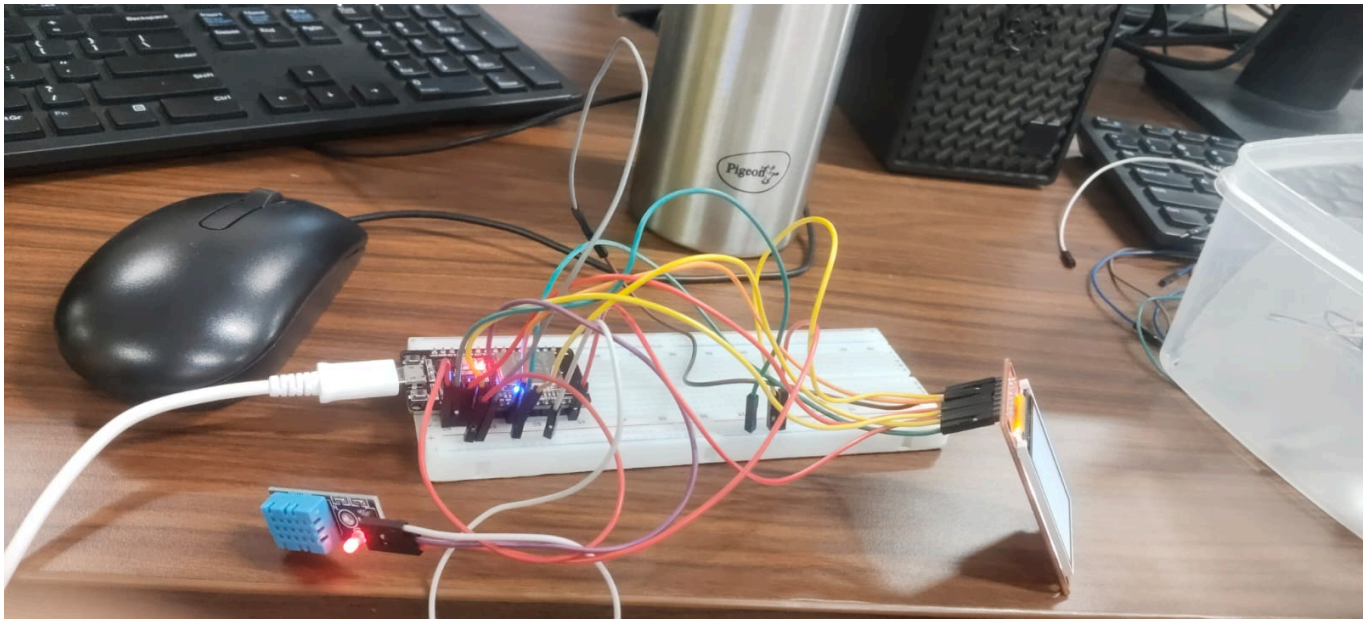
Program

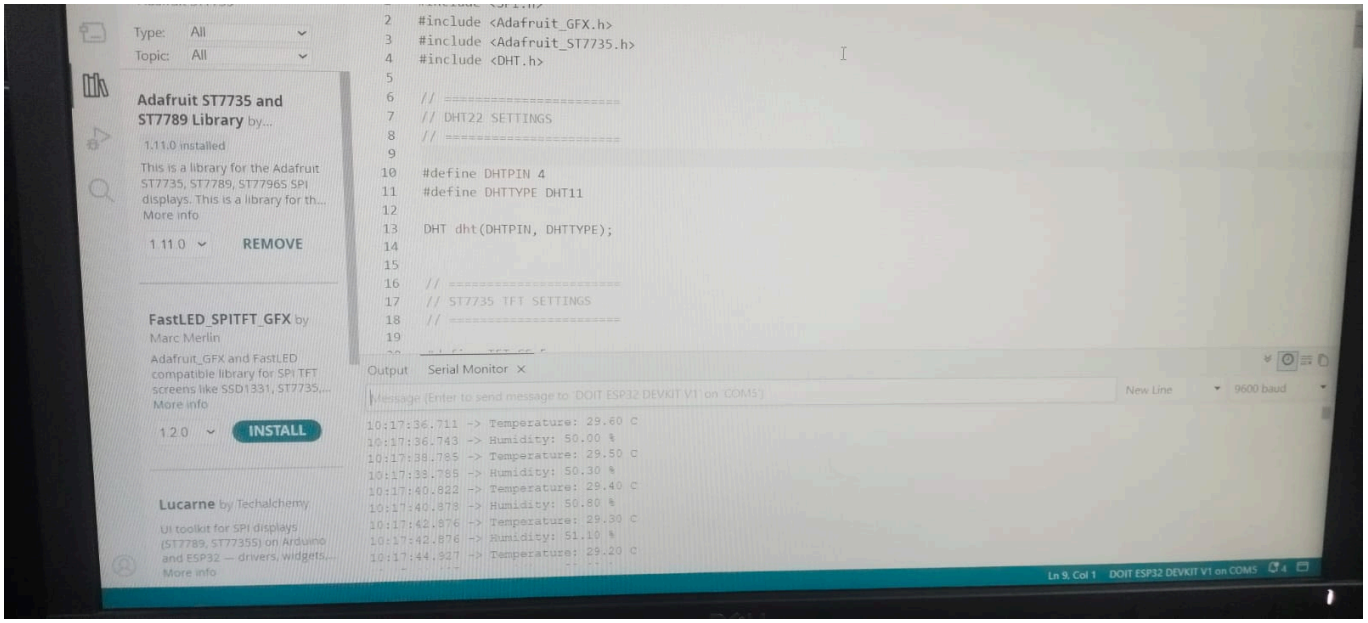
```
#include <SPI.h>
#include <Adafruit_GFX.h>
#include <Adafruit_ST7735.h>
#include "DHT.h"
#define DHTPIN 4
#define DHTTYPE DHT22
#define TFT_CS 5
#define TFT_DC 2
#define TFT_RST 15

Adafruit_ST7735 tft(TFT_CS, TFT_DC, TFT_RST);
DHT dht(DHTPIN, DHTTYPE);
void setup()
{
Serial.begin(9600);
dht.begin();
tft.initR(INITR_BLACKTAB);
tft.setRotation(1);
tft.fillScreen(ST77XX_BLACK);
}
void loop()
{
float t = dht.readTemperature();
float h = dht.readHumidity();
String status;
if (t > 30)
status = "Hot";
else if (t < 18)
status = "Cold";
else
status = "Comfortable";
tft.fillScreen(ST77XX_BLACK);
tft.setTextSize(2);
tft.setTextColor(ST77XX_WHITE);
tft.setCursor(10, 15);
tft.print("Temperature");
tft.setCursor(10, 45);
tft.print(t, 1);
tft.print(" C");
tft.setCursor(10, 75);
tft.print("Humidity");
tft.setCursor(10, 105);
```

```
tft.print(h, 1);
tft.print(" %");

tft.setCursor(10, 135);
tft.print(status);
tft.setTextColor(ST77XX_YELLOW);
tft.setCursor(10, 155);
tft.print("DHT22 + ESP32");
tft.setTextColor(ST77XX_WHITE);
delay(2000);
}
```





Lab Experiment - 3b

IR-Based Object Distance Monitoring System

```
#include <SPI.h>
#include <Adafruit_GFX.h>
#include <Adafruit_ST7735.h>

// ---------- IR SENSOR ----------
#define IR_PIN 34

// ---------- TFT ----------
#define TFT_CS  5
#define TFT_DC  2
#define TFT_RST 15

Adafruit_ST7735 tft(TFT_CS, TFT_DC, TFT_RST);

void setup()
{
 Serial.begin(9600);

 analogReadResolution(12);
 pinMode(IR_PIN, INPUT);

 tft.initR(INITR_BLACKTAB);
 tft.setRotation(1);
 tft.fillScreen(ST77XX_BLACK);
}

void loop()
{
 // Take several readings and average them
 long total = 0;

 for (int i = 0; i < 10; i++)
 {
  total += analogRead(IR_PIN);
  delay(5);
 }

 int raw = total / 10;

 // ESP32 ADC -> voltage
 float voltage = raw * 3.3 / 4095.0;

 // Sharp GP2Y0A21YK approximate distance formula
 float distance = 27.86 / (voltage - 0.42);
```

```
  // Keep distance within sensor's useful range
  if (distance < 10)
    distance = 10;

  if (distance > 80)
    distance = 80;

  // ---------- SERIAL MONITOR ----------
  Serial.print("Raw: ");
  Serial.print(raw);

  Serial.print("  Voltage: ");
  Serial.print(voltage, 2);

  Serial.print(" V  Distance: ");
  Serial.print(distance, 1);

  Serial.println(" cm");

  // ---------- TFT ----------
  tft.fillScreen(ST77XX_BLACK);

  // Title
  tft.setTextSize(2);
  tft.setTextColor(ST77XX_CYAN);
  tft.setCursor(25, 8);
  tft.print("IR SENSOR");

  // Label
  tft.setTextSize(1);
  tft.setTextColor(ST77XX_YELLOW);
  tft.setCursor(40, 40);
  tft.print("DISTANCE");

  // Distance
  tft.setTextSize(2);
  tft.setTextColor(ST77XX_GREEN);
  tft.setCursor(35, 65);
  tft.print(distance, 1);
  tft.print(" cm");

  // Raw value
  tft.setTextSize(1);
  tft.setTextColor(ST77XX_WHITE);
  tft.setCursor(35, 105);
  tft.print("Sensor: ");
  tft.print(raw);

  delay(300);
}
```

Sensor: 4053
10.0 cm
DISTANCE

Lab Experiment-4

ESP32 LDR Light Sensor Alert System

Hardware Connections
LDR to ESP32

LDR Module	**ESP32**
VCC	3.3V
GND	GND
AO	GPIO34

LED and Buzzer

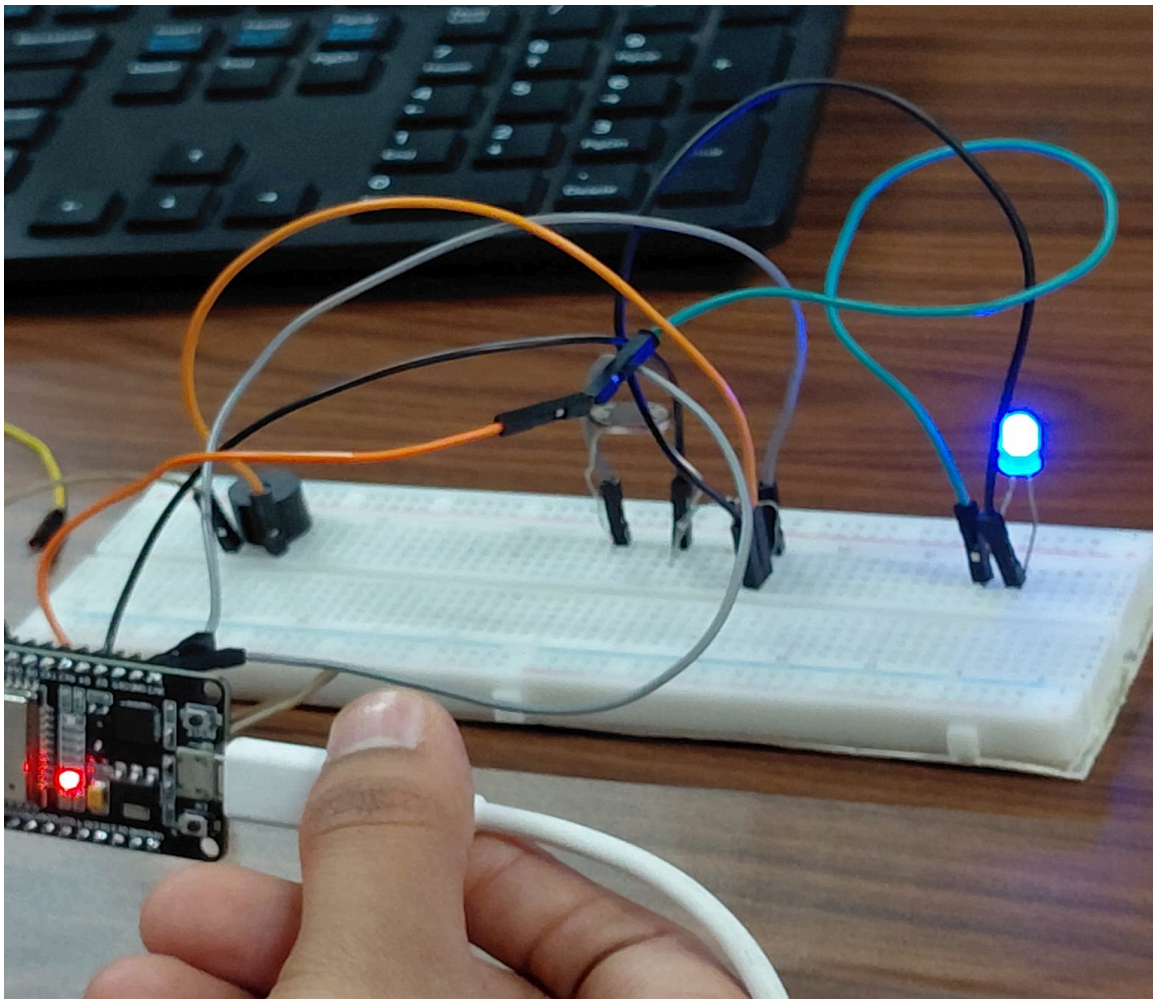
Component	**ESP32**
LED Anode	GPIO25
LED Cathode	GND
Buzzer+	GPIO27
Buzzer-	GND

Program

```
#define LDR 34
#define LED 25
#define BUZZER 27

void setup()
{
Serial.begin(115200);
pinMode(LED, OUTPUT);
pinMode(BUZZER, OUTPUT);
}
void loop()
{
int value = analogRead(LDR);
```

```
Serial.print("LDR = ");
Serial.println(value);
if (value < 1500) // Dark
{
digitalWrite(LED, HIGH);
digitalWrite(BUZZER, LOW);
Serial.println("Dark - LED ON");
}
else // Normal/Bright light
{
digitalWrite(LED, LOW);
digitalWrite(BUZZER, HIGH);
Serial.println("Normal Light - Buzzer ON");
}
delay(500);
}
```



Lab Experiment-5

Pico Relay Dual-Load Switch: Motor & Buzzer Controller

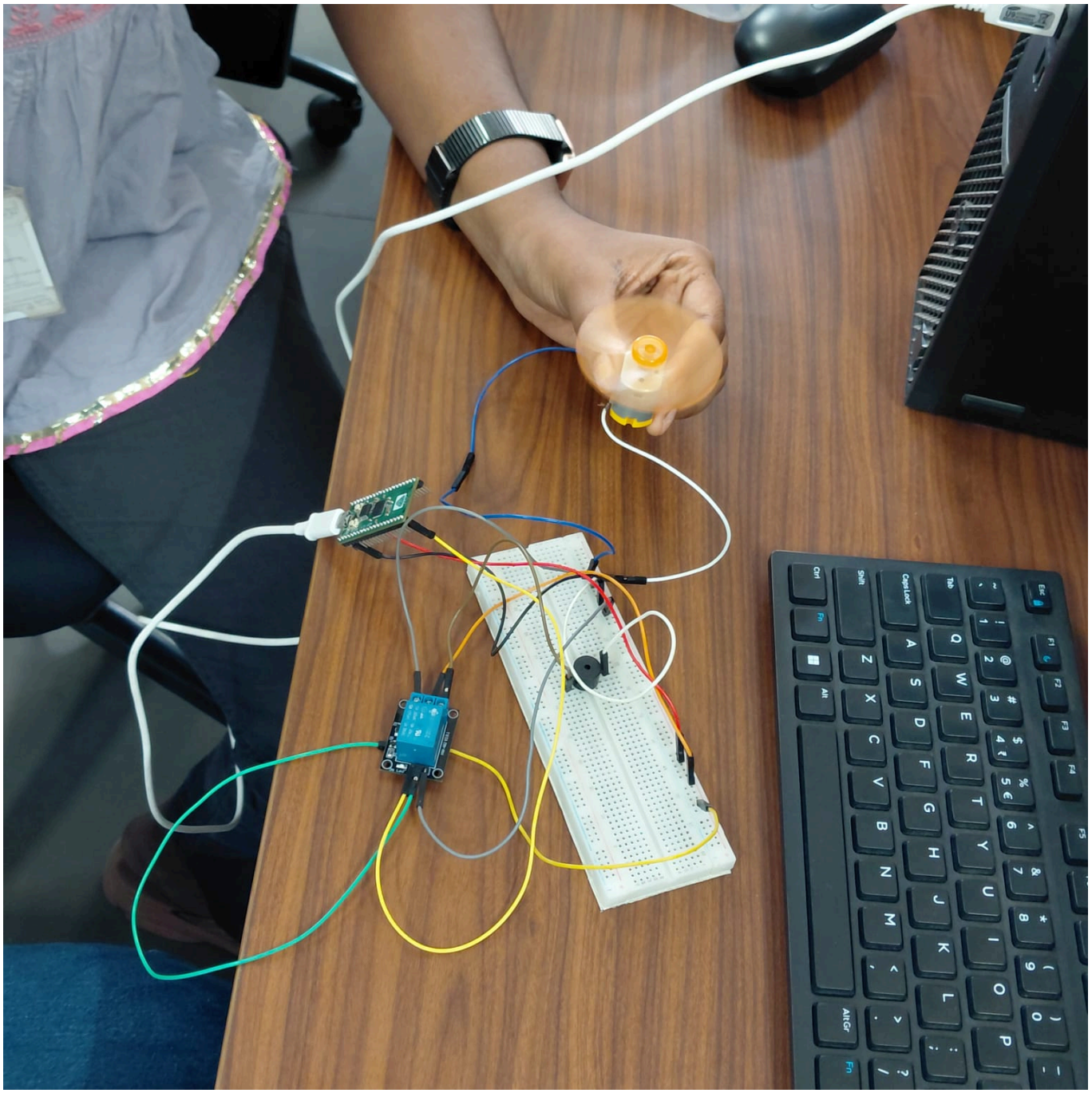
Connection

Component	Connection
Relay VCC	Pico VBUS / 5 V
Relay GND	Pico GND
Relay IN	Pico GP15
Relay COM	External +5 V
Relay NC	Motor+
Relay NO	Buzzer +
Motor -	External GND
Buzzer -	External GND

Program

```
#define RELAY 15
void setup()
{
pinMode(RELAY, OUTPUT);
digitalWrite(RELAY, LOW);
Serial.begin(115200);
}
void loop()
{
// Relay OFF → Motor ON, Buzzer OFF
digitalWrite(RELAY, LOW);

Serial.println("Motor ON - Buzzer OFF");
delay(3000);
// Relay ON → Motor OFF, Buzzer ON
digitalWrite(RELAY, HIGH);
Serial.println("Motor OFF - Buzzer ON");
delay(3000);
}
```



Lab Experiment - 6

a). Experiments using Raspberry Pi / Arduino:
Advanced burglar alarm security system
The objective of this session is to familiarize students with the design and implementation of security and smart automation systems using Arduino/Raspberry Pi and multiple sensors. Students
will learn to develop an advanced burglar alarm system by interfacing a PIR motion sensor, buzzer,
and 4×4 keypad, where the alarm is activated upon motion detection and can be disabled only by
entering the correct password.

Components Required
Arduino Uno
PIR Motion Sensor (HC-SR501)
4×4 Matrix Keypad
Active Buzzer
Breadboard
Jumper Wires
USB Cable

Hardware Connections
PIR Sensor
PIR Sensor Pin Arduino Uno Pin
VCC 5V
GND GND
OUT D2
Active Buzzer
Buzzer Pin Arduino Uno Pin
Positive (+) D3

CS3246– MLOps In Practise

SoCSE, RV University, Bengaluru 24
Negative (-) GND

4×4 Matrix Keypad
Keypad Pin Arduino Uno Pin
Row 1 D4
Row 2 D5
Row 3 D6
Row 4 D7

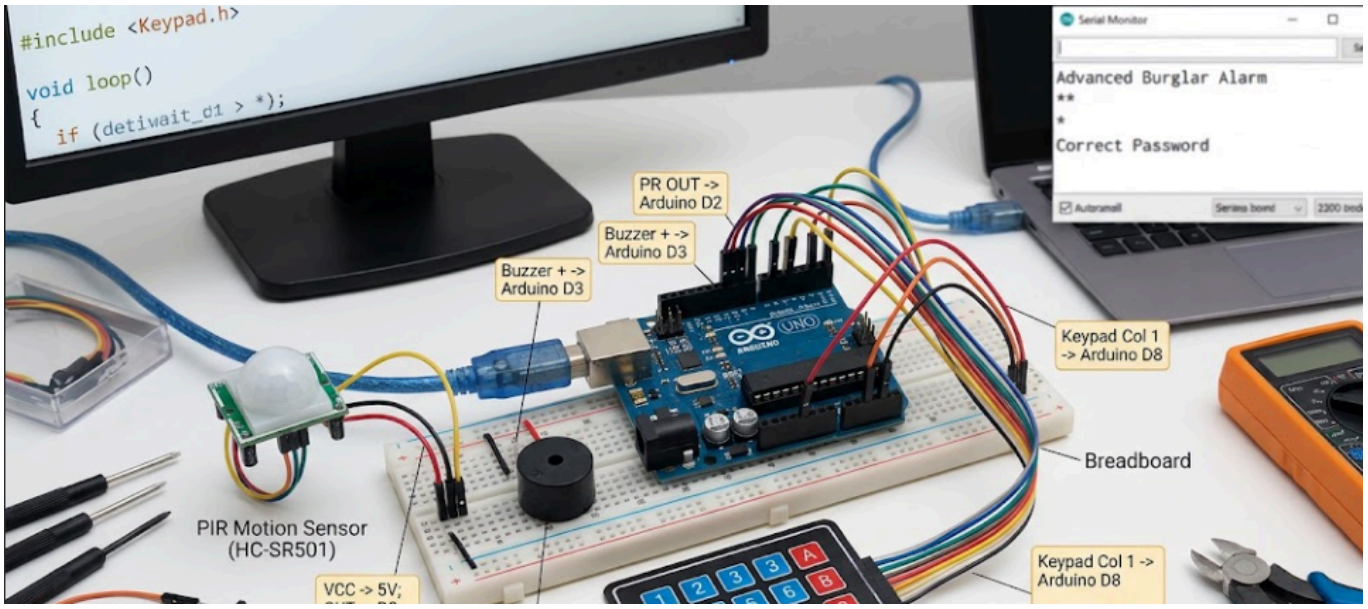
Column 1 D8
Column 2 D9
Column 3 D10
Column 4 D11

Code:

```
#include <Keypad.h>
// PIR and Buzzer Pins
const int pirPin = 2;
const int buzzerPin = 3;
// Password
String password = "1234";
String enteredPassword = "";
// Keypad Configuration
const byte ROWS = 4;
const byte COLS = 4;
char keys[ROWS][COLS] =
{
{'1','2','3','A'},
{'4','5','6','B'},
{'7','8','9','C'},
{'*','0','#','D'}
};
byte rowPins[ROWS] = {4,5,6,7};
byte colPins[COLS] = {8,9,10,11};
Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);
bool alarmActive = false;
void setup()
{
pinMode(pirPin, INPUT);
pinMode(buzzerPin, OUTPUT);
digitalWrite(buzzerPin, LOW);
Serial.begin(9600);
Serial.println("Advanced Burglar Alarm");
}
void loop()
{

// Motion Detection
if (digitalRead(pirPin) == HIGH)
{
alarmActive = true;
}
```

```
// Alarm
if (alarmActive)
{
digitalWrite(buzzerPin, HIGH);
char key = keypad.getKey();
if (key)
{
Serial.print("*");
if (key == '#')
{
if (enteredPassword == password)
{
Serial.println("\nCorrect Password");
digitalWrite(buzzerPin, LOW);
alarmActive = false;
}
else
{
Serial.println("\nWrong Password");
}
enteredPassword = "";
}
else if (key == '*')
{
enteredPassword = "";
Serial.println("\nPassword Cleared");
}
else
{
enteredPassword += key;
}
}
}
else
{
digitalWrite(buzzerPin, LOW);
}
}
```


#include <Keypad.h>
void loop()
{
if (detiwait_01 > *);
Serial Monitor
Advanced Burglar Alarm
**
*
Correct Password
PR OUT -> Arduino D2
Buzzer + -> Arduino D3
Buzzer + -> Arduino D3
Keypad Col 1 -> Arduino D8
Breadboard
PIR Motion Sensor (HC-SR501)
VCC -> 5V;
Keypad Col 1 -> Arduino D8

(b) Automated LED light control based on input from PIR (to detect if people are present) and
LDR (ambient light level).

Hardware Connections

PIR Motion Sensor

PIR Pin	Arduino Uno Pin
VCC	5V
GND	GND
OUT	D7

LDR Sensor Module

LDR Module Pin	Arduino Uno Pin
VCC	5V
GND	GND
AO	A0

LED

LED Pin	Arduino Uno Pin
Anode (+)	D9
Cathode (-)	GND

Code

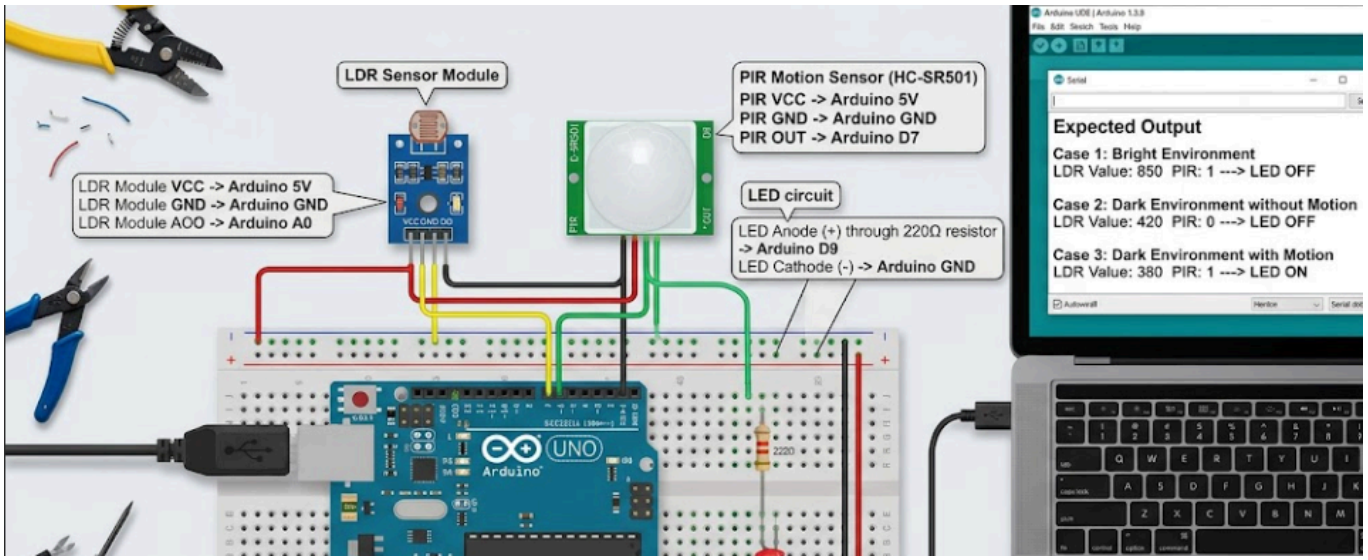
```
const int PIR = 7; // PIR Sensor
const int LDR = A0; // LDR Sensor
const int LED = 9; // LED
int pirState;
int ldrValue;
void setup()
{
pinMode(PIR, INPUT);
pinMode(LED, OUTPUT);
Serial.begin(9600);
}
void loop()
{
// Read sensors
pirState = digitalRead(PIR);
ldrValue = analogRead(LDR);
Serial.print("LDR Value: ");
Serial.print(ldrValue);
Serial.print(" PIR: ");
Serial.print(pirState);
// Dark + Motion Detected
```

```
if ((ldrValue < 700) && (pirState == HIGH))
{
digitalWrite(LED, HIGH);
Serial.println(" --> LED ON");
}
else
{
digitalWrite(LED, LOW);
Serial.println(" --> LED OFF");
}
delay(500);
}
```

Expected Output
Case 1: Bright Environment
LDR Value: 850 PIR: 1 --> LED OFF

Case 2: Dark Environment without Motion
LDR Value: 420 PIR: 0 --> LED OFF

Case 3: Dark Environment with Motion
LDR Value: 380 PIR: 1 --> LED ON



Lab Experiment-7

IOT Framework: Upload humidity & temperature data to Thing Speak, periodically logging ambient light level to Thing Speak.

Components Required

ESP32 Development Board
DHT11 Temperature and Humidity Sensor
Breadboard
Jumper Wires
USB Cable
Wi-Fi Internet Connection

Hardware Connections

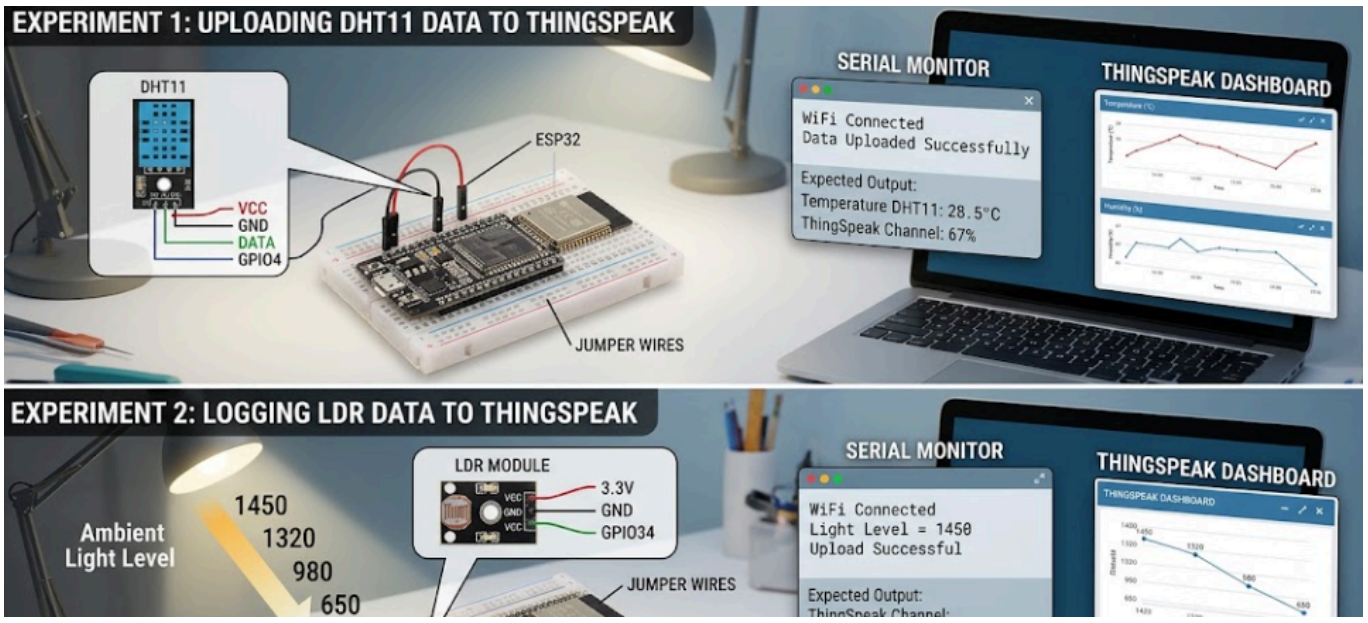
DHT11 Pin ESP32 Pin
VCC 3.3V
GND GND
DATA GPIO4
Note: If using a bare DHT11 sensor, connect a 10 kΩ pull-up resistor between the DATA and VCC
pins.

Code

```
#include <WiFi.h>
#include <ThingSpeak.h>
#include <DHT.h>
#define DHTPIN 4
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);
const char* ssid = "YOUR_WIFI_NAME";
const char* password = "YOUR_WIFI_PASSWORD";
WiFiClient client;
unsigned long channelNumber = YOUR_CHANNEL_ID;
const char * writeAPIKey = "YOUR_WRITE_API_KEY";
void setup()
{
Serial.begin(115200);

dht.begin();
WiFi.begin(ssid,password);
while(WiFi.status()!=WL_CONNECTED)
```

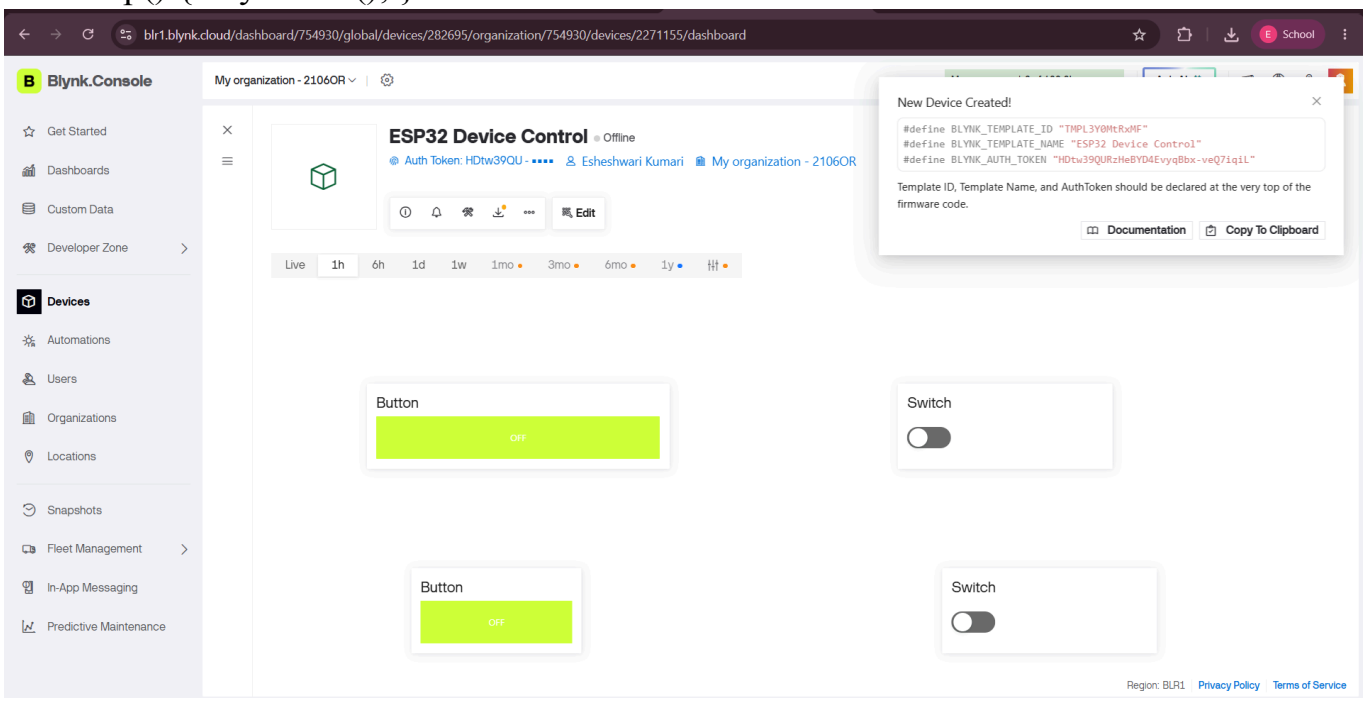
```
{
delay(1000);
Serial.print(".");
}
Serial.println("\nWiFi Connected");
ThingSpeak.begin(client);
}
void loop()
{
float temperature = dht.readTemperature();
float humidity = dht.readHumidity();
if(isnan(temperature) || isnan(humidity))
{
Serial.println("DHT Error");
return;
}
ThingSpeak.setField(1,temperature);
ThingSpeak.setField(2,humidity);
int status = ThingSpeak.writeFields(channelNumber,writeAPIKey);
if(status==200)
Serial.println("Data Uploaded Successfully");
else
Serial.println("Upload Failed");
delay(20000); // ThingSpeak minimum update interval
}
```



Lab Experiment-8

ESP32 Program

```
#define BLYNK_TEMPLATE_ID "TMPL31XXgFINJ"
#define BLYNK_TEMPLATE_NAME "ESP32 Device Control"
#define BLYNK_AUTH_TOKEN HDtw39QURzHeBYD4EvyqBbx-veQ7iqiL
#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
char ssid[] = RVU-Student;
char pass[] = RVU@karnataka;
#define LED 25
#define BUZZER 27
BLYNK_WRITE(V0) { digitalWrite(LED, param.asInt()); }
BLYNK_WRITE(V1) { digitalWrite(BUZZER, param.asInt()); }
void setup() {
pinMode(LED, OUTPUT);
pinMode(BUZZER, OUTPUT);
digitalWrite(LED, LOW);
digitalWrite(BUZZER, LOW);
Serial.begin(115200);
Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
}
void loop() { Blynk.run(); }
```



Lab Experiment-9

HTTP Based: Hosting a basic server from the ESP32 to control various digital based actuators (led, buzzer, relay) from a simple web page.

Components Required
ESP32 Development Board
LED
220 Ω Resistor
5V Relay Module
Active Buzzer
Breadboard
Jumper Wires
USB Cable
Wi-Fi Network
Laptop/Smartphone with Web Browser
Hardware Connections
LED
Component ESP32 Pin
LED Anode (+) GPIO23 (through 220 Ω resistor)
LED Cathode (−) GND

Relay Module
Relay Module Pin ESP32 Pin
VCC VIN (5V)
GND GND
IN GPIO22

Active Buzzer

Buzzer Pin ESP32 Pin
Positive (+) GPIO21
Negative (−) GND

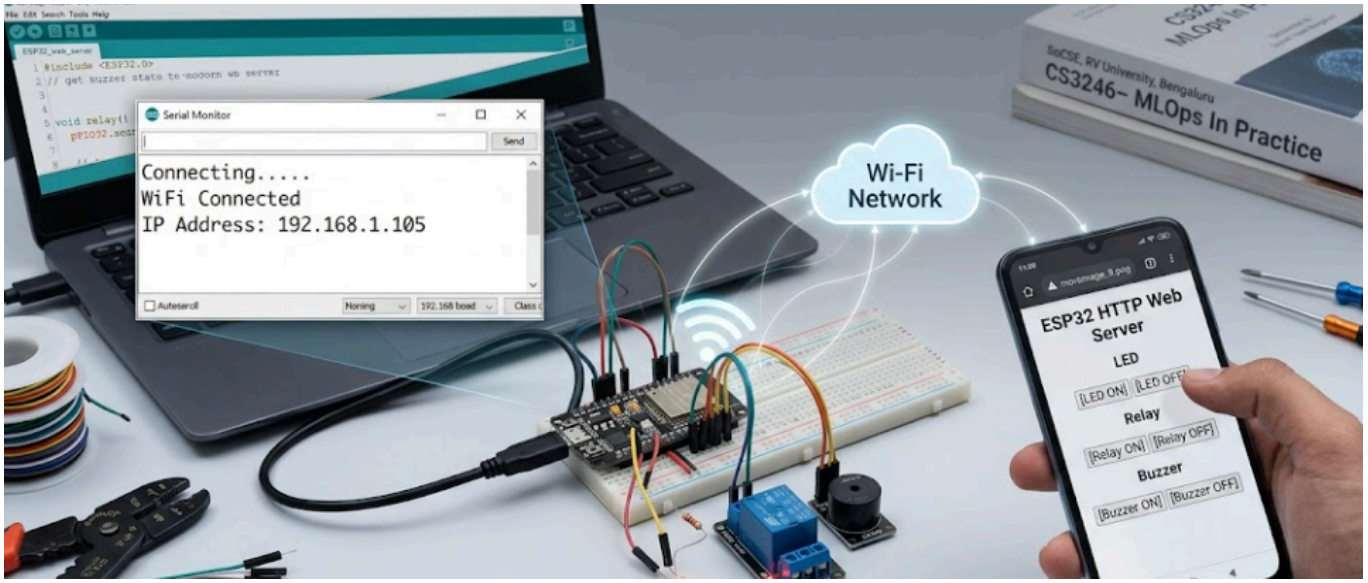
Procedure
Step 1
Install the ESP32 Board Package in the Arduino IDE.
Step 2
Connect the ESP32 board to the computer using a USB cable.
Step 3
Open the Arduino IDE and create a new sketch.
Step 4
Enter your Wi-Fi credentials.

const char* ssid = "YOUR_WIFI_NAME";
const char* password = "YOUR_WIFI_PASSWORD";
Step 5
Compile and upload the program to the ESP32.
Step 6
Open the Serial Monitor at 115200 baud.
Step 7
After connecting to Wi-Fi, note the IP address displayed.
Example
WiFi Connected
IP Address: 192.168.1.105
Step 8
Open any web browser.
Step 9
Type the ESP32 IP address in the browser.
Example
http://192.168.1.105
Step 10
The web page opens with buttons to control the LED, relay, and buzzer.
Code

```
#include <WiFi.h>
const char* ssid = "YOUR_WIFI_NAME";
const char* password = "YOUR_WIFI_PASSWORD";
WiFiServer server(80);
#define LED_PIN 23
#define RELAY_PIN 22
#define BUZZER_PIN 21
void setup()
{
Serial.begin(115200);
pinMode(LED_PIN, OUTPUT);
pinMode(RELAY_PIN, OUTPUT);
pinMode(BUZZER_PIN, OUTPUT);
digitalWrite(LED_PIN, LOW);
digitalWrite(RELAY_PIN, LOW);
digitalWrite(BUZZER_PIN, LOW);
WiFi.begin(ssid, password);
Serial.print("Connecting");
while (WiFi.status() != WL_CONNECTED)
{
delay(500);
Serial.print(".");
}
```

```
Serial.println("\nWiFi Connected");
Serial.print("IP Address: ");
Serial.println(WiFi.localIP());
server.begin();
}
void loop()
{
WiFiClient client = server.available();
if (client)
{
String request = client.readStringUntil('\r');
client.flush();
if (request.indexOf("/LEDON") != -1)
digitalWrite(LED_PIN, HIGH);
if (request.indexOf("/LEDOFF") != -1)
digitalWrite(LED_PIN, LOW);
if (request.indexOf("/RELAYON") != -1)
digitalWrite(RELAY_PIN, HIGH);
if (request.indexOf("/RELAYOFF") != -1)
digitalWrite(RELAY_PIN, LOW);
if (request.indexOf("/BUZZERON") != -1)
digitalWrite(BUZZER_PIN, HIGH);
if (request.indexOf("/BUZZEROFF") != -1)
digitalWrite(BUZZER_PIN, LOW);
client.println("HTTP/1.1 200 OK");
client.println("Content-Type:text/html");
client.println("Connection: close");
client.println();
client.println("<!DOCTYPE html>");
client.println("<html>");
client.println("<head>");
client.println("<title>ESP32 Web Server</title>");
client.println("<style>");
client.println("body{text-align:center;font-family:Arial;background:#f0f8ff;}");
client.println("button{width:140px;height:45px;font-size:18px;margin:8px;}");
client.println("</style>");
client.println("</head>");
client.println("<body>");
client.println("<h2>ESP32 HTTP Web Server</h2>");
client.println("<h3>LED</h3>");
client.println("<a href=\"/LEDON\"><button>LED ON</button></a>");
client.println("<a href=\"/LEDOFF\"><button>LED OFF</button></a>");
client.println("<h3>Relay</h3>");
```

```
client.println("<a href=\"/RELAYON\"><button>Relay ON</button></a>");
client.println("<a href=\"/RELAYOFF\"><button>Relay OFF</button></a>");
client.println("<h3>Buzzer</h3>");
client.println("<a href=\"/BUZZERON\"><button>Buzzer ON</button></a>");
client.println("<a href=\"/BUZZEROFF\"><button>Buzzer OFF</button></a>");
client.println("</body>");
client.println("</html>");
delay(1);
client.stop();
}
}
```



Lab Experiment-10

MQTT Based: Controlling LEDs/Motors from an Android/Web app, Controlling AC Appliances from an android/web app with the help of relay.

Components Required
ESP32 Development Board
LED
220 Ω Resistor
1-Channel Relay Module (or Motor Driver Module)
DC Motor (5V)
External 5V Supply (for motor)
Breadboard
Jumper Wires
USB Cable
Android Smartphone / PC
MQTT Dashboard App (Android) or MQTT Web Client
Wi-Fi Network

Hardware Connections
LED
LED ESP32
Anode (+) GPIO23
Cathode (-) GND

Relay Module

Relay Pin ESP32
VCC VIN (5V)
GND GND
IN GPIO22

DC Motor (via Relay)
Component Connection
External 5V Relay COM
Relay NO Motor Positive
Motor Negative External Supply GND
ESP32 GND

External Supply GND
(Common Ground)

Procedure

Step 1
Install the ESP32 Board Package in the Arduino IDE.
Step 2
Install the following Arduino library:
PubSubClient
Step 3
Connect the ESP32 to the Wi-Fi network.
Step 4
Configure the MQTT broker details:
Broker Address : broker.hivemq.com
Port : 1883
Step 5
Subscribe to the following MQTT topics:
iotlab/led
iotlab/motor
Step 6
Upload the Arduino program to the ESP32.
Step 7
Install an MQTT client (e.g., MQTT Dashboard on Android or MQTTX on PC).
Step 8
Publish commands to control the devices
Topic Message
iotlab/led ON
iotlab/led OFF
iotlab/motor ON
iotlab/motor OFF

Code

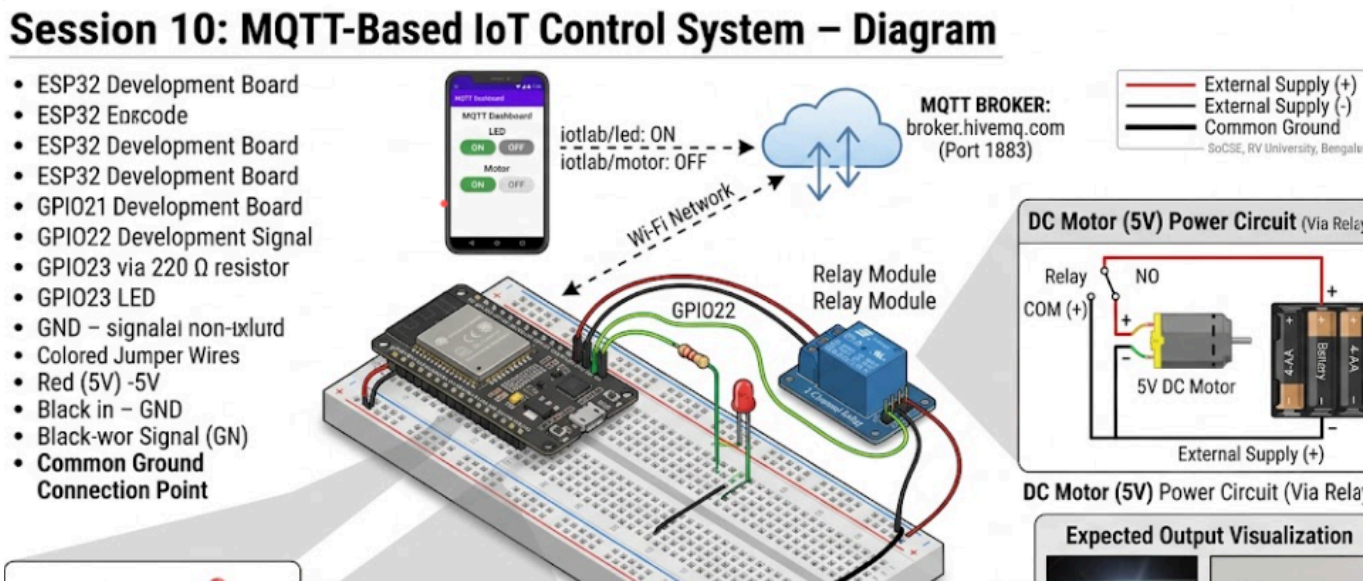
```
#include <WiFi.h>
#include <PubSubClient.h>
const char* ssid = "YOUR_WIFI_NAME";
const char* password = "YOUR_WIFI_PASSWORD";
const char* mqtt_server = "broker.hivemq.com";
WiFiClient espClient;
PubSubClient client(espClient);
#define LED_PIN 23
#define RELAY_PIN 22
void callback(char* topic, byte* payload, unsigned int length)
{
String message = "";
for (int i = 0; i < length; i++)
message += (char)payload[i];
if (String(topic) == "iotlab/led")
```

```
{
if (message == "ON")
digitalWrite(LED_PIN, HIGH);
else
digitalWrite(LED_PIN, LOW);
```



```
}
if (String(topic) == "iotlab/motor")
{
if (message == "ON")
digitalWrite(RELAY_PIN, HIGH);
else
digitalWrite(RELAY_PIN, LOW);
}
}
void reconnect()
{
while (!client.connected())
{
if (client.connect("ESP32_Client"))
{
client.subscribe("iotlab/led");
client.subscribe("iotlab/motor");
}
else
{
delay(2000);
}
}
}
void setup()
{
Serial.begin(115200);
pinMode(LED_PIN, OUTPUT);
pinMode(RELAY_PIN, OUTPUT);
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED)
{
delay(500);
}
```

```
client.setServer(mqtt_server, 1883);
client.setCallback(callback);
}
void loop()
{
if (!client.connected())
reconnect();
client.loop();
}
```



Lab Experiment-11

MQTT Based: Displaying humidity and temperature data on a web-based application.

Components Required

ESP32 Development Board
DHT11 Temperature and Humidity Sensor
Breadboard
Jumper Wires
USB Cable
Wi-Fi Connection
MQTT Broker (e.g., Mosquitto or HiveMQ Public Broker)
Web-Based MQTT Dashboard (Node-RED Dashboard or MQTT Explorer Web)

Hardware Connections

DHT11 Pin ESP32 Pin
VCC 3.3V
GND GND
DATA GPIO4
Note: If using a bare DHT11 sensor, connect a 10 kΩ pull-up resistor between the DATA and VCC
pins.

Steps

Step 1: Install Required Libraries
Install the following libraries in the Arduino IDE:
WiFi
PubSubClient

DHT Sensor Library by Adafruit
Adafruit Unified Sensor
Step 2: Configure MQTT Broker
Example Broker:
Broker Address : broker.hivemq.com
Port : 1883
Step 3: Create MQTT Topics
iotlab/temperature
iotlab/humidity
Step 4: Connect ESP32 to Wi-Fi
Update the Wi-Fi credentials in the Arduino program.
Step 5: Upload the Program
Compile and upload the program to the ESP32.

Step 6: Open the Web Dashboard
Use a web-based MQTT client (such as Node-RED Dashboard or MQTT Explorer Web) and
subscribe to the topics:
iotlab/temperature
iotlab/humidity
The dashboard will display the real-time sensor values.

Code

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <DHT.h>
#define DHTPIN 4
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);
const char* ssid = "YOUR_WIFI_NAME";
const char* password = "YOUR_WIFI_PASSWORD";
const char* mqtt_server = "broker.hivemq.com";
WiFiClient espClient;
PubSubClient client(espClient);
void connectMQTT()
{
while (!client.connected())
{
Serial.print("Connecting to MQTT Broker...");
if (client.connect("ESP32Client"))
{
Serial.println("Connected");
}
```



```
else
{
Serial.print("Failed, State=");
Serial.println(client.state());
delay(2000);
}
}
}
void setup()
{
Serial.begin(115200);
```

```
dht.begin();
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED)
{
delay(500);
Serial.print(".");
}
Serial.println("\nWiFi Connected");
client.setServer(mqtt_server,1883);
}
void loop()
{
if(!client.connected())
{
connectMQTT();
client.loop();
float temperature = dht.readTemperature();
float humidity = dht.readHumidity();
if(isnan(temperature) || isnan(humidity))
{
Serial.println("Sensor Error");
delay(2000);
return;
}
char temp[10];
char hum[10];
dtostrf(temperature,4,2,temp);
dtostrf(humidity,4,2,hum);
client.publish("iotlab/temperature",temp);
client.publish("iotlab/humidity",hum);
Serial.print("Temperature : ");
Serial.print(temp);
Serial.print(" °C ");
Serial.print("Humidity : ");
Serial.print(hum);
Serial.println(" %");
delay(5000);
}
```


MQTT Dashboard
Subscribing to iotlab/temperature
Subscribing to iotlab/humidity
28.5 °C
Temperature
64.2 %
Humidity
Status : Connected
Last Update : 10:35:24
Expected Output:
Connecting to MQTT Broker...
Connected
10Ω

Thank You